

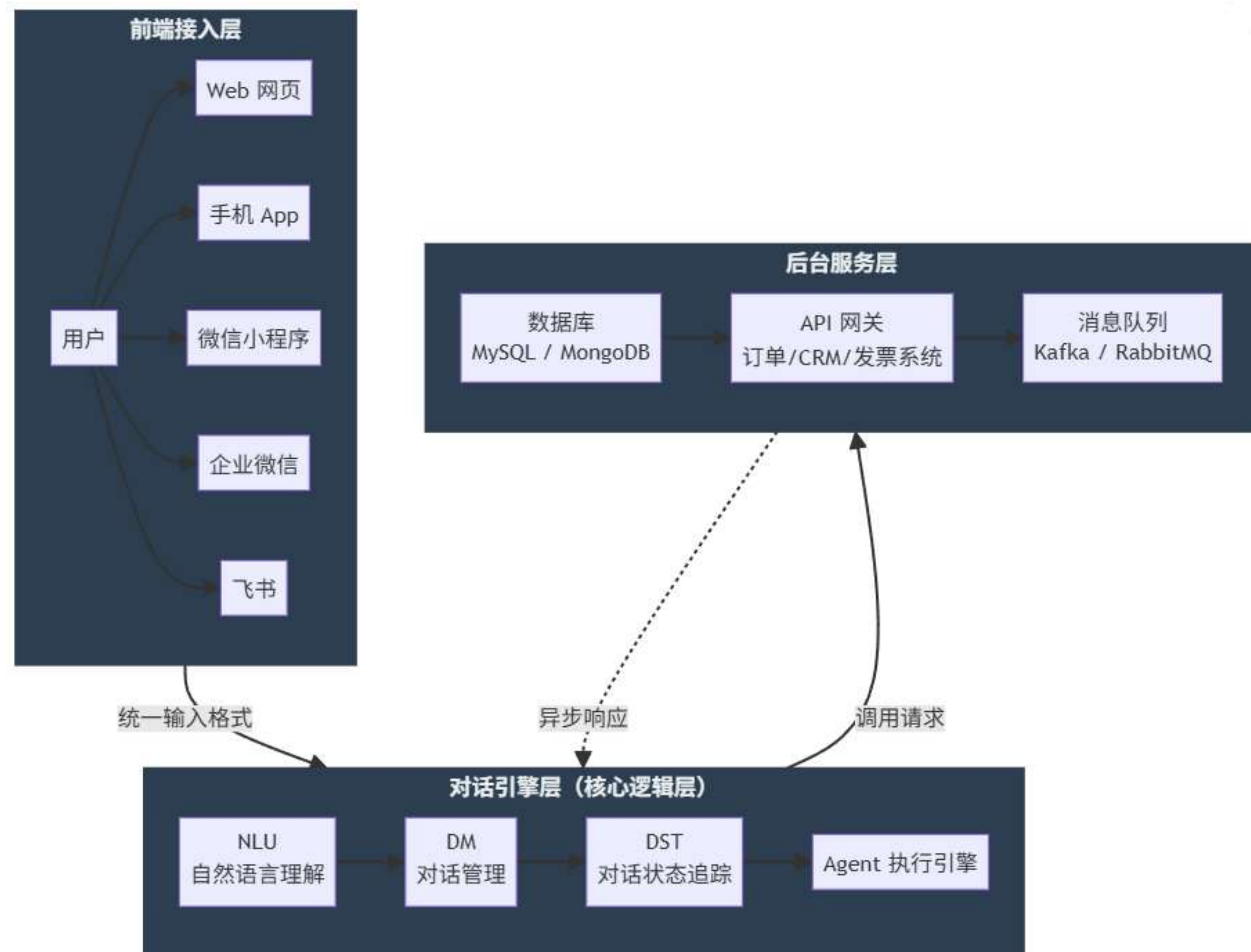
AI 工程化训练营 模块四

尹会生

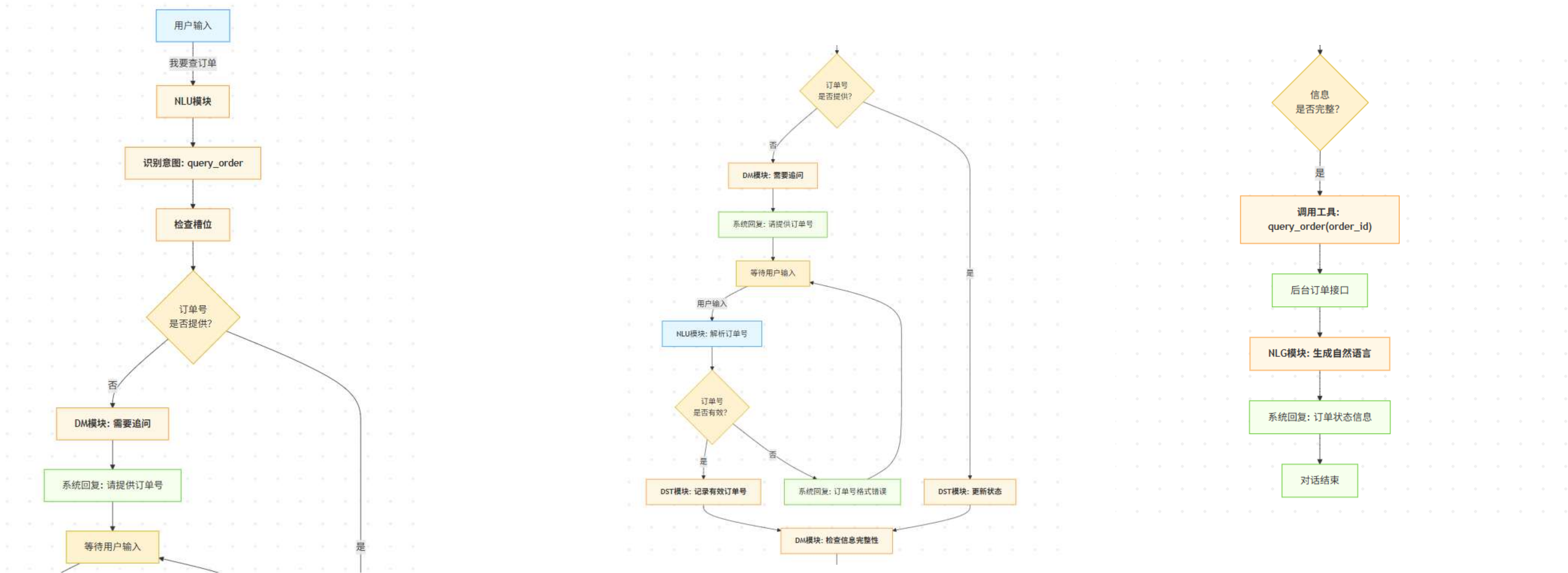
01 搭建基础对话链 —— 对话系统的 “Hello World”

1. 智能对话系统的整体结构

第一层：前端接入层
第二层：对话引擎层
第三层：后台服务层



2. 一个完整对话的处理流程



3. 构建第一个对话链

使用 LangChain 提供的组件，我们可以把提示词、大模型、输出处理连成一条自动流程。

- PromptTemplate（提示词模板）：定义给大语言模型的指令，例如“你是一个客服，请礼貌地回答用户问题”
- ChatModel（聊天模型）：接入大语言模型，如 OpenAI 或通义千问
- StringOutputParser（字符串解析器）：把模型返回的内容转成标准字符串

3. 构建第一个对话链

通过 LCEL 写法，把这些组件连接起来：

```
chain = prompt | llm | StrOutputParser()
```

调用时传入输入：

```
response = chain.invoke({"input": "你好"})
```

这样就完成了一次完整的自动回复。

LangChain 处理链

常见的处理链包含三个 要素

语言模型

提示词模板

输出解释器

使用 LCEL 的好处

1. 简洁语法: 使用管道操作符 | 创建链, 代码直观易读
2. 自动类型推断: 组件间数据类型自动转换, 减少手动处理
3. 并行处理: 内置并行执行能力, 提高处理效率
4. 流式处理: 原生支持流式输出, 适合实时应用
5. 智能缓存: 自动缓存中间结果, 避免重复计算
6. 内置调试: 更好的调试和监控能力
7. 插件化: 支持自定义组件和扩展
8. 标准接口: 统一的 Runnable 接口
9. 版本管理: 支持链的版本控制和管理

4. 封装为网络服务

为了让别人能访问这个对话功能，我们用 FastAPI 把它变成一个网络接口。

- 创建一个 /chat 接口，支持 POST 请求
- 接收参数：用户消息 (message) 和会话编号 (session_id)
- 返回：系统回复内容

实战小任务

实现一个“欢迎语生成器”：

- 输入一个用户姓名，比如“张三”
- 系统返回：“您好，张三！欢迎使用智能客服。”

要求：

- 使用 PromptTemplate 定义提示词
- 使用 LCEL 连接组件
- 通过 FastAPI 提供接口，支持传入 name 参数

深入“输入输出”

提示词模版： 复用大部分静态内容， 只需修改动态变量

掌握自定义模版定义技巧

深入 “输入输出”

大模型接口：

实际工作中，我们会使用私有部署的大模型，因此需要掌握自定义的参数

深入 “输入输出”

输出解释器：

自定义为你需要转换的常见格式。

如：

文本转XML

文本转日期时间

文本转列表

链 -- LangChain的核心概念

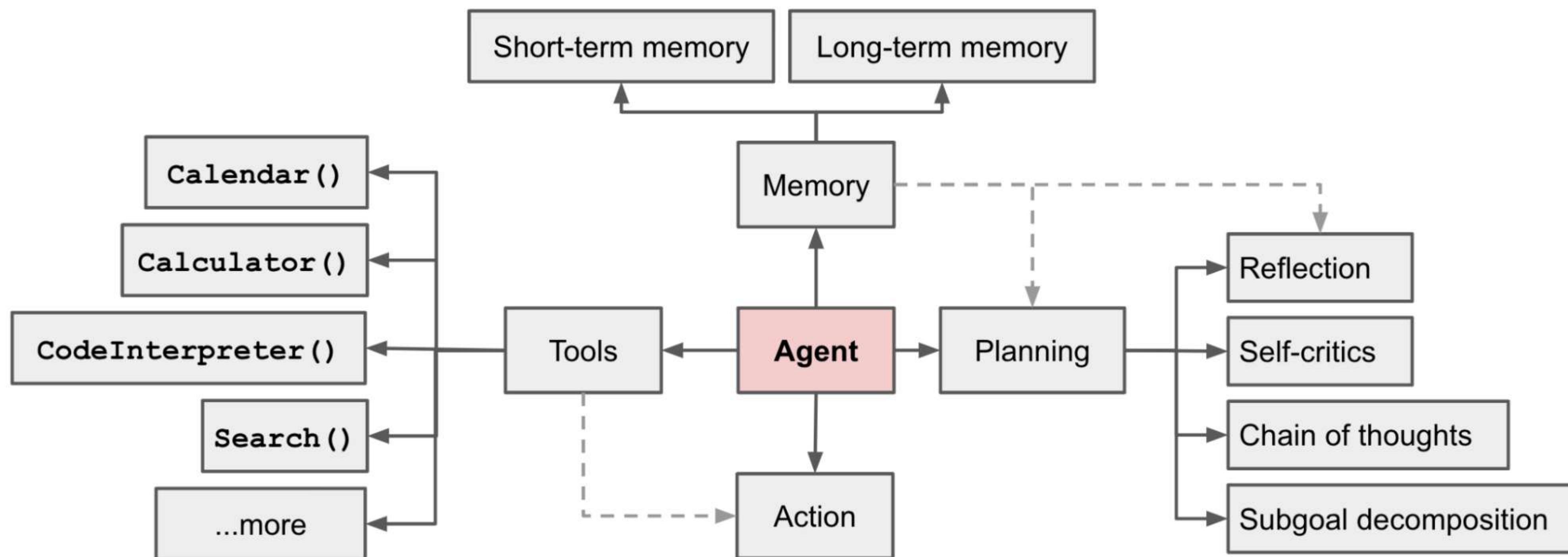
LCEL 语法的核心原理：通过 Python 的操作符重载，让链式调用变得直观和优雅

| 操作符本质上就是在创建一个执行序列，数据从左到右依次流过每个组件。

Runnable 对象接口：LangChain 的整个框架的核心抽象。

02 意图识别流水线设计——让系统“听懂”用户

1. 意图识别与槽位填充



1. 意图识别与槽位填充

意图识别的目标是判断用户想做什么，比如“查订单”“退票”等。

槽位填充则是提取执行该操作所需的参数，比如“订单号”“时间”等。

整个过程分为三个主要方法：

(1) 基于规则的方法 (Rule-based)

适用于表达固定、高频出现的语句，响应速度快，结果可解释。

1. 意图识别与槽位填充

整个过程分为三个主要方法：

(2) 基于机器学习的方法 (ML-based)

适用于语义多样、表达灵活的场景，需要一定量的标注数据进行训练。

1. 意图识别与槽位填充

整个过程分为三个主要方法：

(3) 槽位填充流程

槽位填充是意图识别的后续步骤，目的是提取执行操作所需的参数。

2. 多策略融合的意图识别流水线

(1) 基于大语言模型的意图路由 (LLM Router)

使用大语言模型作为“语义理解专家”，处理模糊、复杂或从未见过的表达。

2. 多策略融合的意图识别流水线

(2) 调用外部机器学习模型 (HuggingFace API)

对接企业内部已有的微调模型，提高识别准确率。

2. 多策略融合的意图识别流水线

(3) 规则引擎（轻量兜底）

实现一个简单的函数，快速判断常见关键词：

2. 多策略融合的意图识别流水线

(4) 多方法融合策略 (Ensemble)

使用 LangChain 的 RunnableParallel 将三种方法并行执行，再通过投票逻辑综合判断结果。

3. 工程增强

为提升系统的稳定性和可维护性，还需加入以下机制：

- RAG 增强识别：从企业 FAQ 库中检索与用户输入相似的问题，辅助判断意图，尤其适用于模糊表达
- 意图白名单控制：只允许返回预设的意图名称，防止大模型随意生成不存在的意图
- Schema 校验：确保槽位结构符合下游工具调用要求，避免参数缺失或类型错误
- 日志记录：记录每种方法的输出结果，便于后续分析偏差、优化策略

实战小任务

构建一个“查订单”意图识别器，要求：

- 使用正则匹配“查订单”“订单号”等关键词
- 调用 HuggingFace 上的微调 BERT 模型进行分类
- 使用大语言模型作为兜底判断（提供 few-shot 示例）
- 提取槽位 order_id（可通过正则或模型提取）
- 输出格式为：`{"intent": "query_order", "slots": {"order_id": "123456"}}`

03 上下文理解与记忆管理 —— 让对话有“记忆”

记忆对 Agent 至关重要

1. 感知能力 (Perception) : 让 Agent 具备环境感知能力, 能接受多模态的信息输入。
2. 决策能力 (Brain-Decision Making) : 让 Agent 具备自主决策和规划的能力, 能够执行更复杂的任务。
3. 记忆能力 (Brain-Memory & Knowledge) : 让 Agent 具备记忆能力, 记忆内部存储了 Agent 的知识和技能。
4. 行动能力 (Action) : 让 Agent 具备与外界交互的能力, 通过行动与感知让 Agent 能自主完成更多复杂任务。

记忆对 Agent 至关重要

1. 让 Agent 具备持续学习能力:能够从过去经历中总结经验以及从错误中学习, 加强任务表现。
2. 让 Agent 能够保持对话的连贯性和行动的一致性:在长对话中能够保持一致的上下文从而保持连贯性。也能避免建立与之前相矛盾的事实, 保持行动的一致性。
3. 让 Agent 能够提供个性化的服务和用户体验:通过历史对话推断用户偏好

知识是静态的, 记忆让 Agent 具备了知识与经验积累和优化的能力。

记忆的划分

1. 按存储时间进行分类:短期记忆 (Short-term Memory) 和长期记忆 (Long-term Memory)
2. 按存储内容进行分类:
 - a. 情境记忆 (Episodic Memory) : 代表经历, 存储了过去发生的事件。
 - b. 语义记忆 (Semantic Memory) : 代表知识, 存储了所了解的知识。
 - c. 流程记忆 (Procedure Memory) : 代表技能, 存储了所掌握的技能。
3. 按内容性质进行分类:可描述 (声明)、不可描述 (声明)

记忆的操作

1. 编码 (Encode) : 获取和处理信息, 将其转化为可以存储的形式。
2. 存储 (Storage) : 在短期记忆或长期记忆中保留编码信息的过程。
3. 提取 (Retrival) : 也可称为回忆, 即在需要时访问并使存储的信息重新进入意识的过程。
4. 巩固 (Consolidation) : 通过巩固将短期记忆转变成长期记忆, 存储在大脑中, 降低被遗忘的可能性。
5. 反思 (Reflection) : 反思是指主动回顾、评估和检查自己的记忆内容的过程, 以增强自我意识, 调整学习策略或优化决策的过程。
6. 遗忘 (Forgetting) : 遗忘是一个自然的过程。

04 让系统“会行动”

1 Agent 的 “感知-决策-执行” 闭环结构

Agent 是连接 “理解” 与 “执行” 的桥梁。

Agent 形成 “感知用户输入 → 决策下一步 → 执行操作” 的闭环，这是智能客服从 “会说话” 到 “能办事” 的关键跃迁。

这一过程涉及到了决策引擎、工具调用和记忆存储三大能力

决策引擎

决策引擎负责决定 Agent 下一步该做什么，常见有两种方式：

1. 基于规则的决策 (Rule-based)

- 使用预设逻辑控制流程。例如：
 - “如果订单金额大于500元，则需人工审核”
 - “如果用户信用等级低，不允许自助退款”
- 优点是逻辑清晰、可解释性强，适合关键流程的约束。

2. 基于大语言模型的决策 (LLM-based)

- 使用 ReAct 模式，让大模型动态生成思考过程和动作指令。
- 支持 “思考 → 动作 → 观察 → 回复” 的循环，适合处理复杂、多变的任务。

工具调用管理器

工具调用管理器统一管理和调度所有可用的工具。

- 支持将多个工具注册到 Agent 中
- 可配置工具的调用顺序：串行或并行
- 提供统一接口进行调用和结果处理

2 Agent 的行为建模

使用 create_react_agent 构建 ReAct 模式 Agent

template = ""Answer the following questions as best you can. You have access to the following tools:

{tools}

Use the following format:

Question: the input question you must answer

Thought: you should always think about what to do

Action: the action to take, should be one of [{tool_names}]

Action Input: the input to the action

Observation: the result of the action

... (this Thought/Action/Action Input/Observation can **repeat N times**)

Thought: I now know the final answer

Final Answer: the final answer to the original input question

Begin!

Question: {input}

Thought:{agent_scratchpad}""

3 工具定义与调用实现

(1) 定义工具 (Tool)

使用 @tool 装饰器定义可调用的功能接口。

@tool

```
def query_order(order_id: str) -> dict:  
    """查询订单状态"""  
    return db.query(order_id)
```

(2) 参数自动提取

通过 Function Calling 或结构化输出 (Structured Output) 机制，让大模型自动从用户输入中提取参数，如订单号、时间等。

(3) 错误处理机制

- 工具调用失败时，自动重试或提示用户
- 对非法参数进行校验，防止无效调用
- 设置超时机制，避免长时间阻塞

实战小任务

实现以下功能：

用户说“查一下订单1234567890”，系统通过 Agent 自动识别意图，调用 query_order 工具，获取订单状态和物流信息，并返回自然语言回复。

要求：

- 使用 create_react_agent 构建 Agent
- 注册 query_order 工具
- 支持参数自动提取
- 显示完整的调用过程（思考 → 动作 → 观察 → 回复）

05 用 LangGraph 实现多轮对话流程控制

Agent 的挑战

- 多轮对话
- 状态管理
- 工作流程控制

LangGraph 是基于 LangChain 生态的智能体框架，采用图架构设计，将智能体的工作流程抽象为由节点和边构成的有向有环图。

LangGraph 关键组件

节点 (Node)：编码代理逻辑的 Python 函数。它们接收当前的 State 作为输入，执行一些计算或副作用，并返回一个更新的 State。

- 大模型
- 工具
- 业务逻辑
- 子图

边 (Edge)：Python 函数，根据当前的 State 决定接下来执行哪个 Node。它们可以是条件分支或固定转换。

- 入口边
- 普通边
- 条件边

状态 (State)：一个共享数据结构，表示应用程序的当前快照。它可以是任何 Python 类型，但通常是 TypedDict 或 Pydantic BaseModel。

- 上下文与持久化
- 节点数据

LangGraph 工作原理

通过组合 Nodes 和 Edges, 您可以创建复杂、循环的工作流, 使 State 随时间演进。
然而, 真正的力量来自于 LangGraph 如何管理 State。

需要强调的是: Nodes 和 Edges 不过是 Python 函数——它们可以包含一个 LLM, 也可以只是普通的 Python 代码。

简而言之: **节点完成工作, 边指示下一步该做什么。**

与 LangChain 的差异

特性	LangChain (LCEL)	LangGraph
核心架构	链式，有向无环图 (DAG)	图架构，有向循环图 (DCG)
设计理念	简化 LLM 应用开发，线性流程	构建复杂、状态化的智能体系统
状态管理	相对简单，短期记忆	强大的状态管理和持久化机制
工作流程	线性或分支流程，相对固定	循环流程，支持迭代和动态分支
适用场景	简单应用，快速原型开发	复杂智能体，多智能体系统

LangGraph 的图

langgraph 图是对一个工作流的抽象，由多个节点和边组成。

节点是实际干活的函数，边负责传递消息。

图的执行会被分解为“超步”（super-step）。

图的节点可以串行也可以并行。

组合 LangGraph 基础能力

借助快照机制，对状态进行查看和控制

```
graph.get_state_history(config)
```

借助条件边，增加 Langgraph 的健壮性

可视化与调试

1. 借助 python Notebook 进行可视化

```
from IPython.display import Image, display
```

```
display(Image(graph.get_graph().draw_mermaid_png()))
```

2. 借助 Langgraph Studio 完成可视化

```
pip install -U "langgraph-cli[inmem]"
```

```
langgraph dev
```


06 工具调用引擎设计

工具的注册与发现机制

1. 插件化加载 importlib

通过 Python 的 importlib 模块，从指定目录动态导入工具文件。

该机制支持在不重启服务的情况下新增工具文件，实现热扩展。

实战任务

实现以下功能：

用户说“查一下订单1234567890”，系统通过 Agent 自动识别意图，调用 query_order 工具，获取订单状态和物流信息，并返回自然语言回复。

要求：

- 工具支持参数格式校验（如订单号长度）
- 调用失败时最多重试3次
- 查询结果缓存5分钟
- 输入非法时返回明确错误提示

07 热更新机制

实现零停机升级

- 模型指针替换：切换模型时不中断服务
- 插件动态重载：新增功能无需重启
- 健康检查与回滚机制：更新失败可自动恢复

1. 模型热更新

将模型部署为远程服务，通过 API 调用，便于版本切换。

```
def get_model_client(model_name="intent-bert-v1"):
    return HuggingFaceClient(model_name)
```

更新时只需修改配置中的 `model_name`，即可指向新版本模型（如 v2），实现无感切换。

优点：部署简单，支持灰度发布；缺点：依赖网络，延迟较高。

2. 插件热重载

文件监听 + 动态重载

使用 watchdog 库监听插件目录变化。

版本校验与灰度发布

为防止新版本引入错误，加入安全机制：

- 加载前检查 config.yaml 是否符合预期格式
- 验证新插件的工具接口是否兼容
- 支持按用户 session ID 或请求头进行灰度发布（例如仅对10%流量启用新功能）

3. 健康检查与回滚机制

(1) 健康检查接口

提供一个 /health 接口，用于检测系统状态。

可用于负载均衡器探活或自动化监控。

(2) 回滚机制

当新模型或插件加载失败时，自动回退到旧版本。

- 保留旧模型引用，在新模型验证失败时重新启用
- 插件加载异常时记录日志并保持原有功能不变
- 提供手动触发回滚的管理接口

项目实践

阶段一：基础对话系统搭建

使用 LangChain 构建基础 Chain: Prompt → LLM → OutputParser

用户说“我昨天下的单”，系统能结合当前时间推断“昨天”的具体日期

阶段二：多轮对话与工具调用

实现“订单查询”“退款申请”等多轮交互流程，支持工具自动调用。

使用 LangGraph 构建以下流程：

- 用户说“查订单” → 追问“请提供订单号”
- 收到订单号后 → 调用 `query_order(order_id)` 工具
- 返回订单状态与物流信息

项目实践

阶段三：热更新与生产部署

实现模型与插件的热更新，完成系统部署与监控。

1. 模型热更新
2. 插件热重载
3. 暴露健康检查接口 /health
4. 编写自动化测试脚本
 - 测试“发票开具”插件的功能正确性
 - 验证热更新后旧会话不受影响